

# AJP simulation 手動BO運用・計算手法ガイド

AJP simulation workspace

2026 年 9 月 10 日

## 1 本版の目的と、ここまでの流れ

本書はmainブランチの正本である。AJP ノズルの 11 設計変数から structured メッシュを生成し、定常 CFD、有限半径粒子追跡、3 目的 2 制約の評価、次候補提案を行う。候補を作る・計算へ投入する・結果を回収する・次へ進むという判断を利用者が一段ずつ行う。

これまで、圧力緩和と CFD 完了判定の修正、structured メッシュへの一本化、potentialFoam の SIGFPE 修正、residualControl の導入を行った。2026 年 9 月 8 日に有限半径の壁面直接さえぎりを追加し、9 月 9 日に利用者・SSH 環境の設定を整理した。9 月 10 日に、自動版と手動版を同時運用できるようブランチを分けた。

| 版    | 運用と保管内容                                                                                                   |
|------|-----------------------------------------------------------------------------------------------------------|
| main | 本書の手動版。各段階を利用者が実行する。実行に必要なコードと本ガイドを保持。                                                                    |
| 自動版  | feature/structured-finite-radius-mobo-v2。複数ホストの常駐 dispatcher、自動投入・回収・反復、および旧資料を保持。分岐時点の commit は 1a04973。 |

両版は別ディレクトリへ clone する。実行中の clone でブランチを切り替えると、ジョブが参照するスクリプトも変わる。手動版の既定出力先は campaigns/manual\_structured\_finite\_radius/ である。自動版の case・観測 CSV を共有しない。過去の点粒子モデルの観測も、新しい有限半径モデルの学習へ直接混ぜない。

## 2 利用者が変わるときの設定

計算用クラスタには本人の SSH ユーザーでログインし、そのクラスタの clone 内で操作する。本版はホスト間 SSH を制御しないため、SSH のユーザー名はログイン時に指定する。Python・OpenFOAM などの利用者固有値は site.env へ集約する。

```
cp site.env.example site.env
${EDITOR:-vi} site.env
source ./site.env
"$AJP_PYTHON_BIN" tools/check_site_config.py
```

| 設定項目                | 意味                                         |
|---------------------|--------------------------------------------|
| AJP_PYTHON_BIN      | NumPy・Gmshが利用できる Python。計算ノードから見える絶対パスを推奨。 |
| AJP_OPENFOAM_BASHRC | 使用する OpenFOAM の etc/bashrc。                |
| AJP_PARTICLE_SOLVER | 粒子 solver のパス。空なら OpenFOAM 環境の PATH から探索。  |
| AJP_SLURM_NODELIST  | 投入先 node。空なら Slurm が選択。                    |
| AJP_WORKER_ENV      | 通常は空。旧 worker-env.sh の読み込みを無効化。            |

site.env は Git 管理外であり、bo\_loop.py が読み込んで Slurm ジョブへ渡す。自動版とは設定項目が異なるため、手動版の site.env.example から新しく作成する。共有する形状範囲・目的関数・出力先は bo\_config.json で管理する。Slurm が選ぶ各 node から、clone、Python、OpenFOAM と粒子 solver を利用できる必要がある。

OpenFOAM と Slurm が導入された環境で、次を実行する。

```
"$AJP_PYTHON_BIN" -m pip install numpy gmsh
bash tools/build_solver.sh "$AJP_OPENFOAM_BASHRC" \
"$PWD/vendor/aerosoldynamicsFoam"
"$AJP_PYTHON_BIN" tools/check_site_config.py --local-tools

# 有効観測 32 件以上から BO 提案する前に導入
"$AJP_PYTHON_BIN" -m pip install -r requirements-mobo.txt
```

build スクリプトは同梱粒子 solver と finiteRadiusDeposition を作成する。実行 solver と同じ OpenFOAM・利用者環境で build する。OpenFOAM の版が異なる環境へ共有ライブラリの binary を流用しない。

### 3 手動 BO の操作手順

以下は clone 直下での例である。init と suggest は候補生成だけを行い、ジョブを投入しない。最初は少数 case で全段階を確認する。

#### 3.1 初期候補と CFD

```
"$AJP_PYTHON_BIN" bo_loop.py --config bo_config.json \
init --initial-batch 32
"$AJP_PYTHON_BIN" bo_loop.py --config bo_config.json status

"$AJP_PYTHON_BIN" bo_loop.py --config bo_config.json \
submit-cfd --cases case_0000 case_0001 --dry-run
"$AJP_PYTHON_BIN" bo_loop.py --config bo_config.json \
submit-cfd --cases case_0000 case_0001
```

--dry-run で投入内容を確認してから実際に投入する。squeue -u "\$USER"、status と各 case のログを確認し、CFD の成功後に粒子へ進む。残差停止で 10,000 反復前に終了した場合も、実際の最新 CFD 時刻の流れ場を粒子へ渡す。

#### 3.2 粒子追跡と回収

```
"$AJP_PYTHON_BIN" bo_loop.py --config bo_config.json \
submit-particles --cases case_0000 case_0001 --dry-run
"$AJP_PYTHON_BIN" bo_loop.py --config bo_config.json \
submit-particles --cases case_0000 case_0001

# 粒子計算の終了を確認してから回収
"$AJP_PYTHON_BIN" bo_loop.py --config bo_config.json \
collect --cases case_0000 case_0001
"$AJP_PYTHON_BIN" bo_loop.py --config bo_config.json status
```

collect は particle\_fates\_all.csv を評価し、campaign 内の bo\_observations.csv へ反映する。--cases を省略した回収は campaign 全体を対象とする。ログと評価値を確認し、同じ手順で残りの初期候補を評価する。

### 3.3 次候補と終了判断

```
"$AJP_PYTHON_BIN" bo_loop.py --config bo_config.json \  
suggest --batch-size 8
```

有効観測が 32 件未満なら空間充填 maximin、32 件以上なら制約付き qLogNEHVI で候補を提案する。学習点上限 384、候補 pool 2048、Monte Carlo sample 64 が現在の設定である。次に投入する case、失敗後の再試行、次 batch へ進む時点、計算予算と終了条件は利用者が判断する。自動の予算停止・停滞停止は本版にはない。

status の cfd\_failed または particle\_failed はログを確認する。再試行は回収前に原因を修正し、該当段階の\*\_FAILED 印だけを取り除いてから再投入する。失敗として観測に残す場合は collect する。強制終了では失敗印が残らない場合もあり、Slurm の sacct でも確認する。squeue を確認できないときは投入を停止する。

## 4 計算手法

### 4.1 設計変数と structured メッシュ

11 設計変数の現在の範囲を示す。派生寸法と幾何制約は bo\_loop.py で計算する。

| 変数                     | 範囲       | 内容           |
|------------------------|----------|--------------|
| R_mm                   | 0.5–2.0  | 基準半径 [mm]    |
| alphaRin               | 0.3–0.7  | 内側入口半径比      |
| alphaL0                | 1.0–5.0  | 上流長さ比        |
| L1_mm                  | 1.0–20.0 | 軸方向長さ [mm]   |
| alphaL2                | 0.5–5.0  | 下流長さ比        |
| alphaL3                | 0.0–1.0  | 追加長さ比        |
| theta0_deg, theta1_deg | 各 10–60  | 傾斜角 [deg]    |
| alphaTheta2            | 0.0–1.0  | 第 3 角度比      |
| alphanat               | 0.5–5.0  | 厚さ比          |
| U_mps                  | 1–50     | 基準入口速度 [m/s] |

全長  $\leq 120$  mm、 $L_2 \leq 70$  mm、 $l_2, l_{2y}, R_3 \leq 45$  mm を課す。base/meshGen2.py は断面を block へ分割し、 $y$  軸まわりの 5 度、一層 wedge の structured メッシュを生成する。通常の上限は 120,000 nodes、360,000 elements である。unstructured 版は本版の選択肢に含めない。

### 4.2 搬送流 CFD

Gmsh から gmshToFoam で変換し、potentialFoam -initialiseUBCs -writep で初期場を作る。-initialiseUBCs は入口流量境界を評価するために必要であり、全域ゼロ速度のまま圧力近似へ進んだ旧版の SIGFPE への修正である。初期化の失敗は CFD 失敗として扱う。

搬送流は非圧縮・定常 simpleFoam、RAS モデル kOmegaSST で解く。入口は flowRateInletVelocity で wedge 角に対応する流量を与え、固定壁は noSlip、前後面は wedge とする。圧力は field 緩和 0.3、 $U, k, \omega$  は equation 緩和 0.7、初期値は  $k = 10^{-4}$ 、 $\omega = 1$  である。

residualControl は  $p < 10^{-5}$ 、 $U < 10^{-4}$ 、 $k, \omega < 10^{-5}$  を使い、最大 10,000 反復で停止する。solver の終了状態、ログの End、最大速度を検査し、CFD\_DONE または CFD\_FAILED を記録する。 $\max |U| \leq 10,000$  m/s という上限は数値異常の guard であり、物理的妥当性や十分な収束を保証する条件ではない。

### 4.3 粒子追跡と有限半径のさえぎり

vendor/aerosolDynamicsFoam の basicKinematicCloud による、一方向連成の Lagrangian 追跡を用いる。現在の力モデルは sphereDrag で、搬送流速度を cellPoint 補間する。粒径は 0.1, 0.2, 0.3, 0.5, 0.8, 1, 2, 3, 5, 7  $\mu\text{m}$ 、各 bin 100 位置で合計 1000 parcel である。

解析的位置追跡を有効にし、時間刻み上限  $10^{-4}$  s、集計 chunk 0.1 s とする。95% の fate が確定するか、次の設計別上限へ到達すると停止する。

$$T_{\max} = \min(5 \text{ s}, \max(0.5 \text{ s}, 3T_{\text{transit}})) .$$

既定の 4 並列は mesh の領域分割ではなく、各 rank が full mesh を持ち、注入粒子を 4 shard へ分ける方式である。

標準の `standardWallInteraction/stick` による中心軌道の衝突に加え、`finiteRadiusDeposition` が移動区間と固定壁面の距離を調べる。有効半径は

$$r_{\text{eff}} = f_r d_p + \delta, \quad f_r = 0.5, \quad \delta = 0$$

であり、粒子表面が初めて壁に触れる点で沈着とする。候補 face を空間検索し、距離と二分探索で最初の接触位置を求め、patch、粒径、粒子 ID、中心と壁面座標を記録して parcel を除去する。対象は `wallSubstrate`, `wallUpper`, `wallDown`, `wallCavity` である。各 shard のイベントを `particle_fates_all.csv` へ統合する。

これは固定壁への幾何学的な直接さえぎりである。先に堆積した粒子による遮蔽、堆積層の成長、粒子同士の衝突・凝集、二方向連成、Brownian/random force と乱流分散は含まない。

#### 4.4 3 目的と 2 制約

注入数を  $N_{\text{inj}}$ 、基板中心  $(x, z) = (0, 0)$  の半径  $5 \mu\text{m}$  円内へ到達した数を  $N_{\text{target}}$ 、基板到達総数を  $N_{\text{substrate}}$ 、他の 3 壁への沈着数を  $N_{\text{wall}}$  とする。次の 3 目的を最大化する。

$$f_1 = N_{\text{target}}/N_{\text{inj}}, \quad (1)$$

$$f_2 = \begin{cases} N_{\text{target}}/N_{\text{substrate}}, & N_{\text{substrate}} > 0, \\ 0, & N_{\text{substrate}} = 0, \end{cases} \quad (2)$$

$$f_3 = \frac{5}{5 + r_{90}[\mu\text{m}]} \min\left(1, \frac{N_{\text{target}}}{20}\right). \quad (3)$$

$r_{90}$  は基板到達粒子の、target 中心からの距離の 90% 分位である。基板到達粒子がないときは  $f_3 = 0$  とする。制約は

$$N_{\text{resolved}}/N_{\text{inj}} \geq 0.95, \quad N_{\text{wall}}/N_{\text{inj}} \leq 0.05$$

である。有限半径の追加は壁面・基板沈着数と目的値を変え得るため、物理設定と objective version を観測に対応させて保管する。

## 5 検証状況と、本計算前に残る作業

過去の検証では structured の端点 12 形状と random 10 形状の mesh 検査、代表 case の CFD 安定化、修正後 `potentialFoam` の正常終了を確認した。有限半径については OpenFOAM v2406 での build/load と、serial での強制 1000 接触イベントの記録・除去を確認した。これらは分岐前に行った検証の記録であり、本版によるクラスタ上の新たな本番計算の実績ではない。

本版のソース回帰テストは次で実行する。

```
python3 -m unittest discover -s tests -v
```

ソースの回帰テストと設定診断は、実 solver・並列実行・物理モデルの妥当性の検証を代替しない。本計算前には次を確認する。

1. 使用する各実行環境で solver と有限半径 library を build し、少数 case で CFD 投入、粒子投入、fate 回収、目的評価までを通す。
2. 同一 case で `radiusFactor=0` と 0.5、serial と 4 shard を比較する。粒子収支、ID の重複、接触距離と patch 分類を確認する。
3. structured メッシュ密度と粒子時間刻みの感度を確認する。
4. residual 停止と固定反復で粒子目的値を比較し、CFD 停止閾値を確定する。
5. 有限半径法で初期観測を新たに取得し、方法・設定・ソース revision を記録して BO へ進む。

手動版と自動版の同時運用では、出力先を分離したうえで同じ Slurm 資源の総使用量も確認する。新しい調査は独立した重複報告を増やさず、本書に条件・結論・根拠となる case を追記する。

## 6 ファイルの役割

| 場所                  | 本版での役割                         |
|---------------------|--------------------------------|
| bo_loop.py, mobo.py | 手動操作、候補生成、観測評価、多目的 BO。         |
| bo_config.json      | 共通の計算条件と campaign 設定。          |
| site.env.example    | 利用者固有設定の雛形。site.env は Git 管理外。 |
| base/               | structured メッシュと CFD テンプレート。   |
| baseparticle/       | 粒子追跡、有限半径接触、集計のテンプレート。         |
| vendor/             | 粒子 solver の build 用ソースとライセンス。  |
| tools/              | 設定診断、solver build などの補助。       |
| tests/              | 軽量のソース回帰テスト。                   |
| reports/            | 本書の TeX/PDF 正本。                |
| campaigns/          | 実行時の case・ログ・観測。Git 管理外。       |

常駐 dispatcher、複数ホスト設定、旧資料と archive/は自動版ブランチで参照する。大容量の生成 case、個人設定、LaTeX 中間生成物は GitHub へ登録しない。